

Computer Vision Lecture 4

Machine Learning Intro (Part 2)

Jens Rittscher



Learning objectives

- ML overview
- Model validation
- Dimensionality reduction methods
- Classification using Gaussian mixtures
- Linear models
- Support vector machines
- Neural networks

Classification

Bayes Rule & Bayesian Concepts in Machine Learning

Given the probabilities of a k given classes we wish to make a decision

$$p(\mathcal{C}_k|x) = \frac{p(x|\mathcal{C}_k)p(\mathcal{C}_k)}{p(x)}$$

The posterior summarises everything we know about the unknown quantities

Maximum a posteriori (MAP) estimates

It is obtained by maximising the posterior distribution:

$$\operatorname{argmax}_{\mathcal{C}_k \in \mathcal{C}} P(\mathcal{C}_k | x)$$

Like any other point estimate the MAP estimate does not provide any measure of uncertainty.

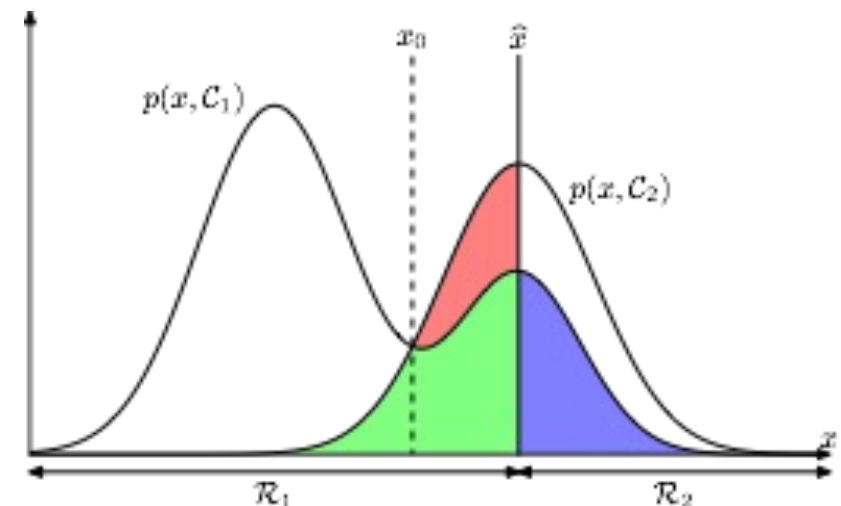
The MAP estimate can result in overfitting!

Classification errors

Errors arise from the red, green and blue regions.

Red, green: points are from class C_2 but are classified as C_1

Blue: points are from class C_1 but are classified as C_2



Bayes risk

The probability of an misclassification error is given by

$$\begin{aligned} p(\text{mistake}) &= p(x \in \mathcal{R}_1, \mathcal{C}_2) + p(x \in \mathcal{R}_2, \mathcal{C}_1) \\ &= \int_{\mathcal{R}_1} p(x, \mathcal{C}_2) dx + \int_{\mathcal{R}_2} p(x, \mathcal{C}_1) dx \end{aligned}$$

Linear models

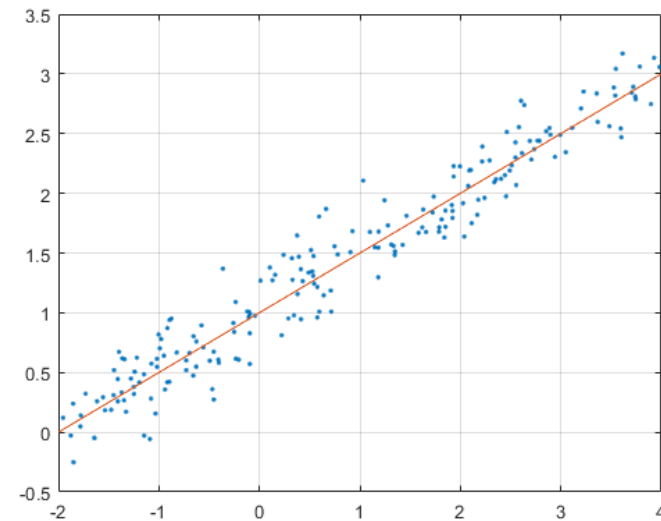
$$y_i = a_0 + a_1 x_{i,1} + \cdots + a_M x_{i,M} + \epsilon_i$$

$$y_i = a_0 + \mathbf{x}^\top \mathbf{a} + \epsilon$$

$a_0, \dots, a_M$ coefficients / weights

Estimated from the training data

ϵ_i Independent identically
distributed 0-mean Gaussian
noise



Linear models

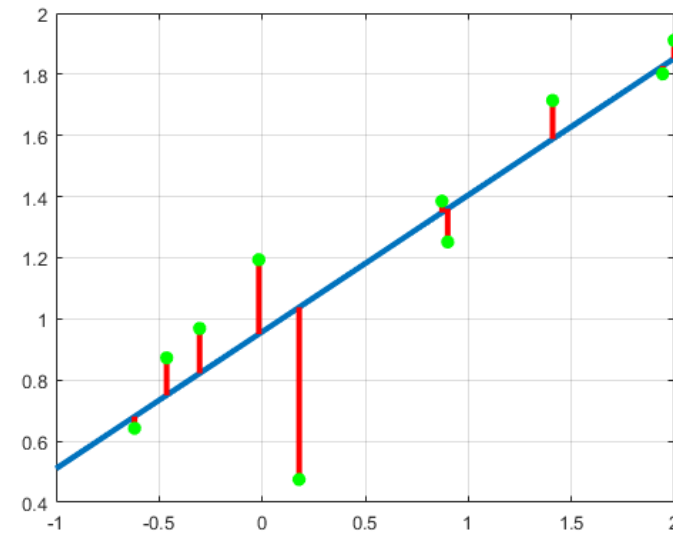
Estimated function:

$$\hat{y}_i = \hat{a}_0 + \mathbf{x}^\top \hat{\mathbf{a}}$$

Loss:

$$L(y_i, \hat{y}_i)$$

Measures how well the estimated model fits the expected output.



Least squares

Minimising the loss: $\hat{\mathbf{a}} = \arg \min_{\mathbf{a}} \sum_i (y_i - f_{\mathbf{a}}(x_i))^2$

Gradient: $\frac{\partial l}{\partial \mathbf{a}} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{a})$

Least squares estimate: $\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,M} \\ \vdots & \vdots & & \vdots \\ 1 & x_{N,1} & \dots & x_{N,M} \end{bmatrix} \quad \mathbf{a} = \begin{bmatrix} a_0 & a_1 & \dots & a_M \end{bmatrix}$$

Classification

Goal: Learn a mapping from inputs to outputs

$$x \longrightarrow y \quad \text{with} \quad y \in \mathcal{C} := \{1, \dots, C\}$$

Probabilistic predictions for a given model M :

$$p(y|x, M, \mathcal{D})$$

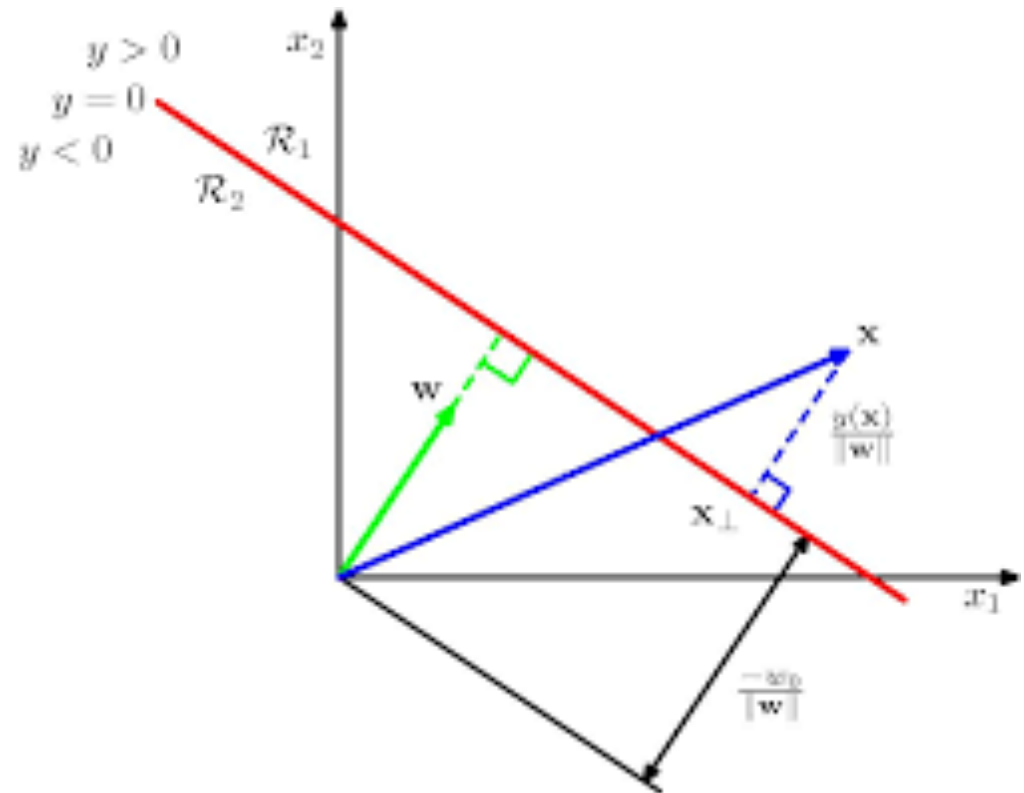
Maximum a posteriori estimate for the class label y :

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{C}} p(y = c|x, M, \mathcal{D})$$

Linear models for classification

A linear model can now be used for classification.

$$y(x) = w^T x + w_0$$



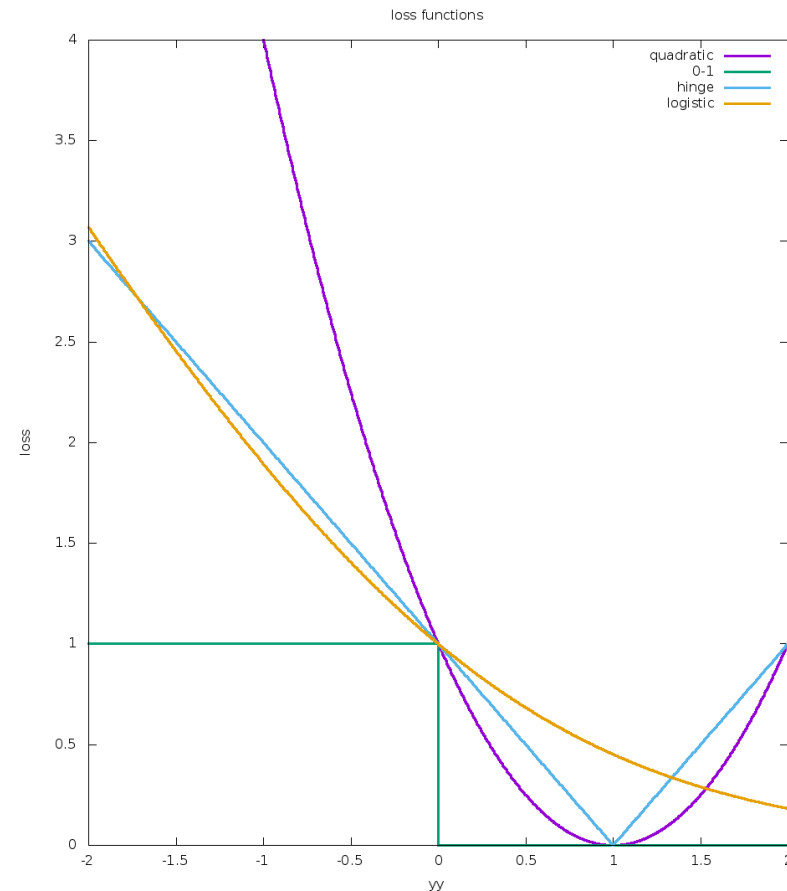
Loss functions

Quadratic loss: $(y_i - \hat{y}_i)^2$

0-1 loss: $I(y_i \neq \hat{y}_i)$

Hinge loss: $\max(0, 1 - y_i \hat{y}_i)$

Logistic loss: $\log_2(1 + e^{-y_i \hat{y}_i})$

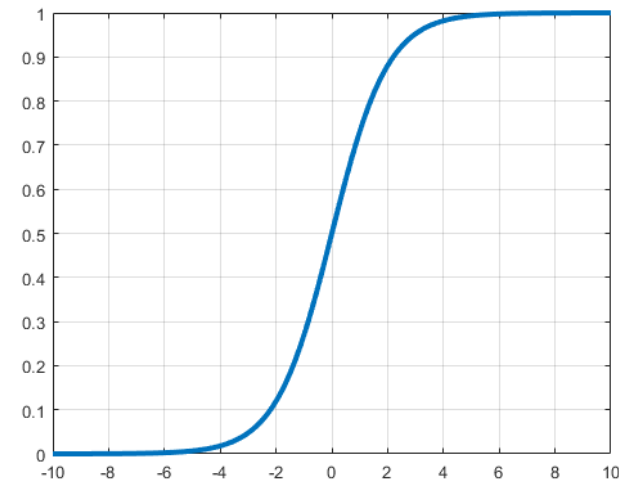


Logistic regression

We now use the logistic sigmoid function acting on a linear function of the feature vector to define

$$P(C_1|x) = \sigma(w^T x)$$

$$P(C_2|x) = 1 - P(C_1|x)$$



$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

Logistic regression

For a data set

$$\{x_i, y_i\} \quad \text{with} \quad y_i \in \{0, 1\}$$

We can now write the likelihood function as

$$p(y|w) = \prod_{i=1}^N p(C_1|x_i)^{y_i} \{1 - p(C_1|x_i)\}^{1-y_i}$$

Logistic regression

The log likelihood can now be written as

$$E(w) = -\ln p(y|w) = -\sum_{i=1}^N y_i \ln p(C_1|x_i) + (1-y_i) \ln(1 - p(C_1|x_i))$$

Because

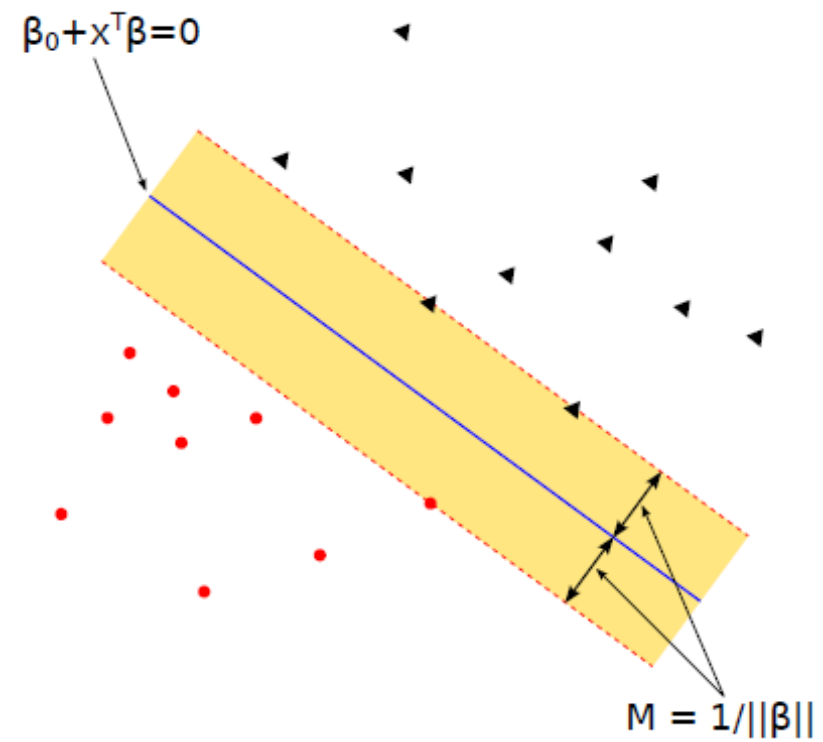
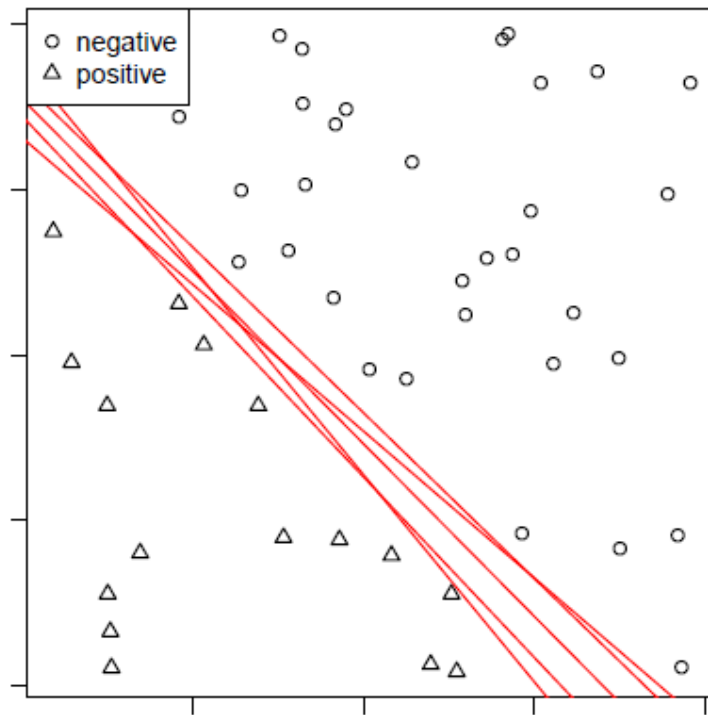
$$\frac{d\sigma}{da} = \sigma(1 - \sigma)$$

$$\nabla E(w) = \sum_{i=1}^N (p(C_1|x_i) - y_i)x_i$$

Support Vector Machines

- We now generalize the technique of using linear decision boundaries for classification
- We consider cases where the classes are nonseparable and do overlap

Margin classifiers



optimal hyperplane
separation

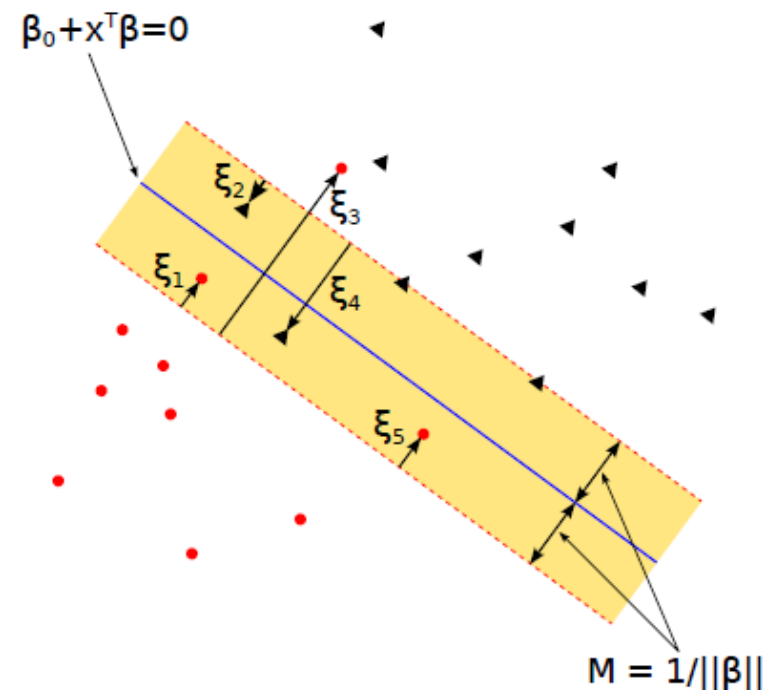
Support vector machines (SVMs)

Optimal decision boundaries for overlapping classes

$$(\hat{a}_0, \hat{\mathbf{a}}) = \arg \min_{a_0, \mathbf{a}} \sum_{n=1}^N \xi_n + \lambda \|\mathbf{a}\|^2$$

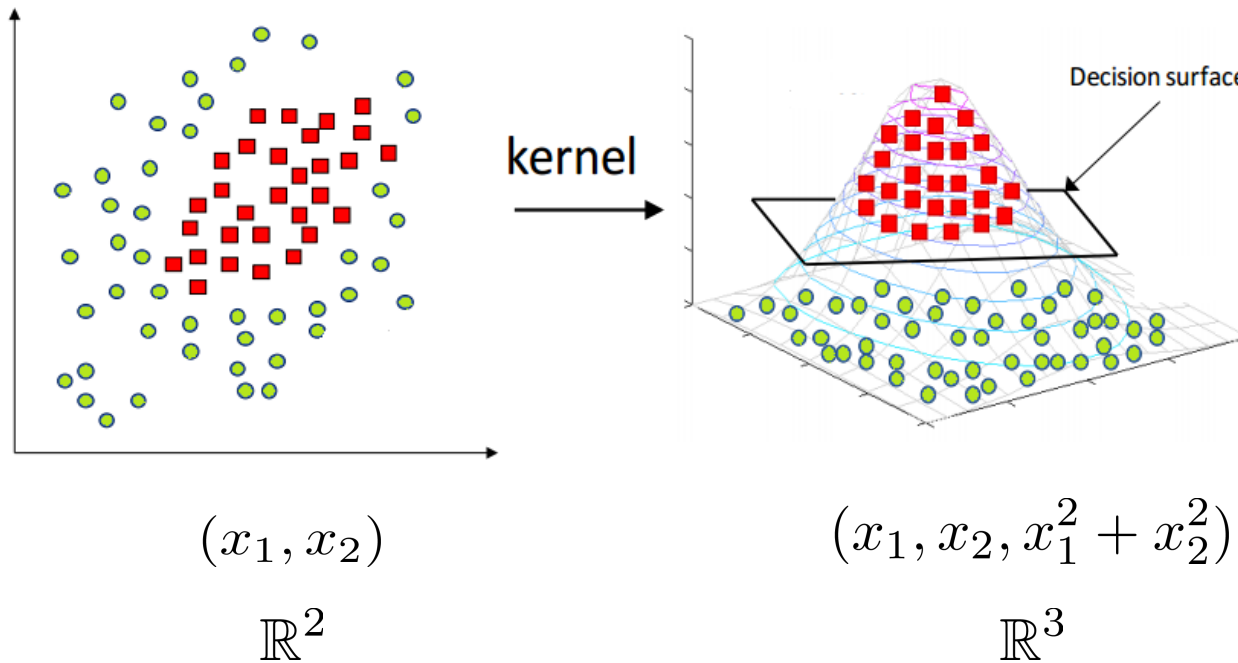
$$\text{s. t. } \begin{cases} \xi_n \geq 0 \\ y_n(a_0 + \mathbf{x}_n^\top \mathbf{a}) \geq 1 - \xi_n \end{cases}$$

ξ_n slack variables



Non-linear SVMs

We cannot separate the original data using a hyperplane. – But after mapping the data it is possible to achieve this.



Non-linear SVMs

Kernel functions

Linear

$$K(\mathbf{x}_k, \mathbf{x}_l) = \mathbf{x}_k^\top \mathbf{x}_l$$

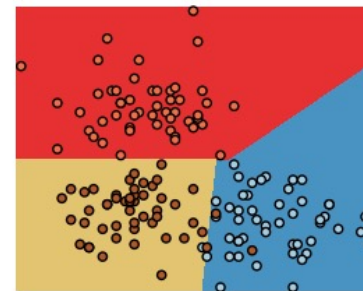
Polynomial

$$K(\mathbf{x}_k, \mathbf{x}_l) = (\mathbf{x}_k^\top \mathbf{x}_l + c)^d$$

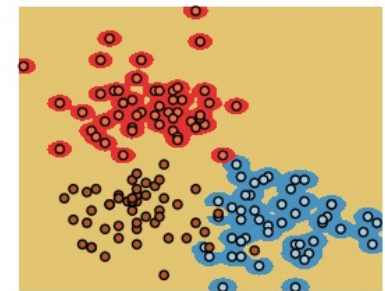
Gaussian (or radial basis function)

$$K(\mathbf{x}_k, \mathbf{x}_l) = \exp\left(-\frac{\|\mathbf{x}_k - \mathbf{x}_l\|^2}{2\sigma^2}\right)$$

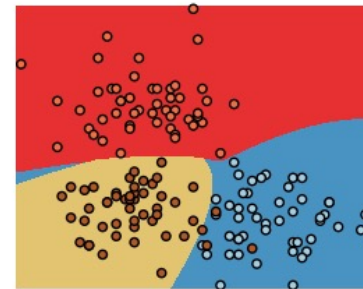
SVC with linear kernel



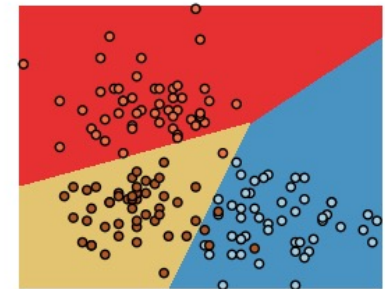
SVC with RBF kernel



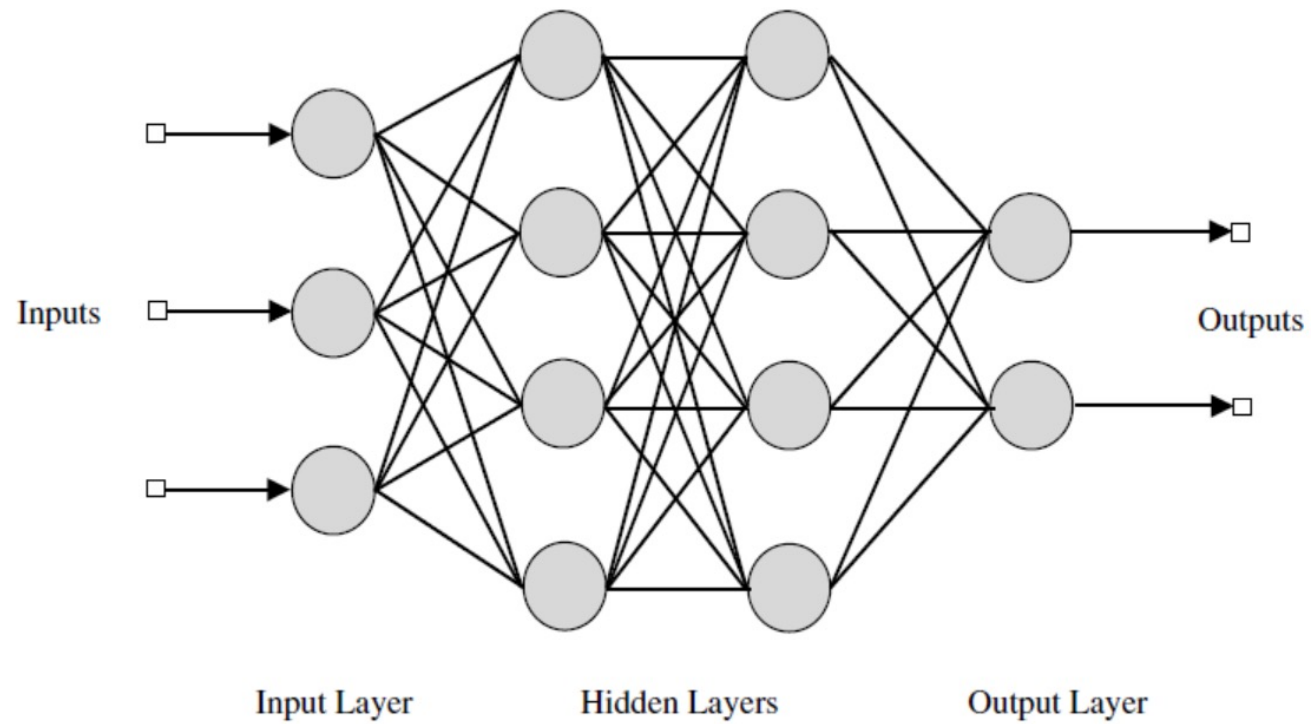
SVC with polynomial (degree 3) kernel



LinearSVC (linear kernel)



Feed-forward neural networks



After Bishop, Pattern Recognition & Machine Learning, Chapter 5

Feed-forward Network functions

The linear models for regression and classification we discussed before were of the form:

$$y(x, w) = f \left(\sum_j^M w_j \phi_j(x) \right)$$

We now extend this model by introducing basis function that depend on parameters we learn from training data.

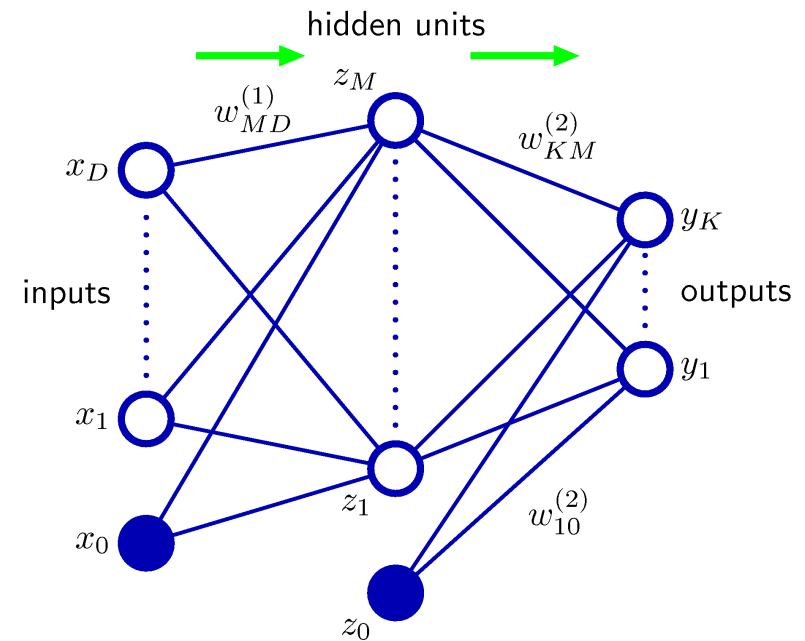
Input layer

Linear combination of the inputs

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}$$

Non-linear activation function

$$z_j = h(a_j)$$



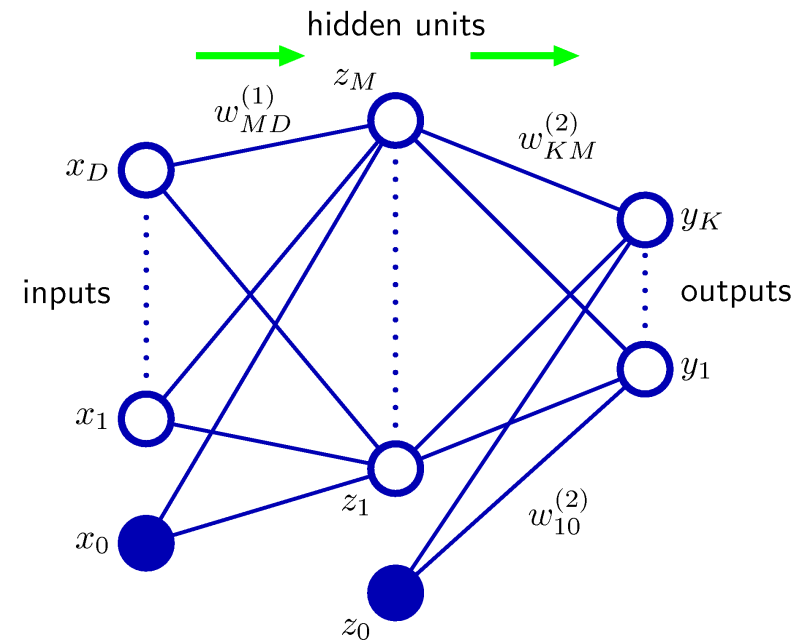
Hidden units

Linear combination of the outputs from the input layer

$$a_k = \sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)}$$

Again, use a non-linear transformation to generate the outputs

$$y_k = \sigma(a_k) \quad \sigma(a) = \frac{1}{1 + \exp(-a)}$$

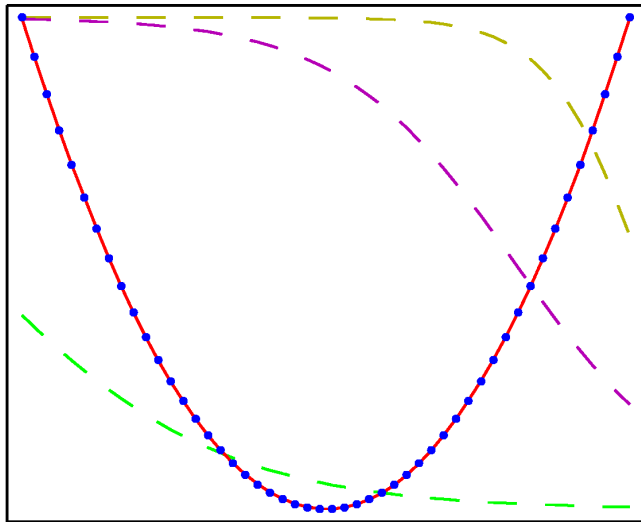


Overall network function

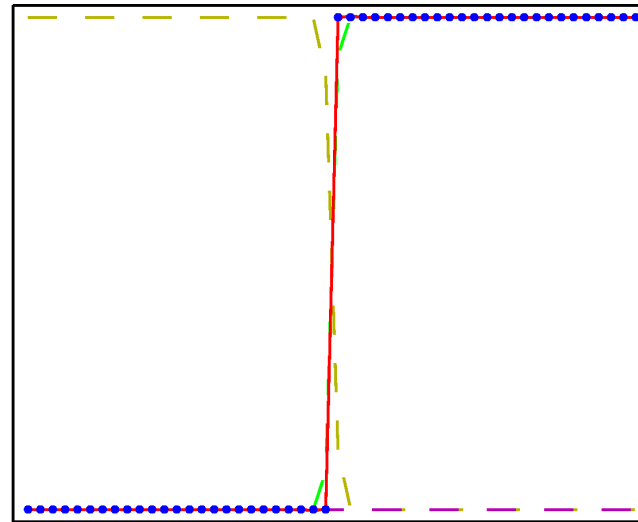
$$y_k(x, w) = \sigma \left(\sum_{j=1}^M w_{kj}^{(2)} h \left(\sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)} \right) + w_{k0}^{(2)} \right)$$

We can extend this function by adding additional hidden layers.

Approximation properties

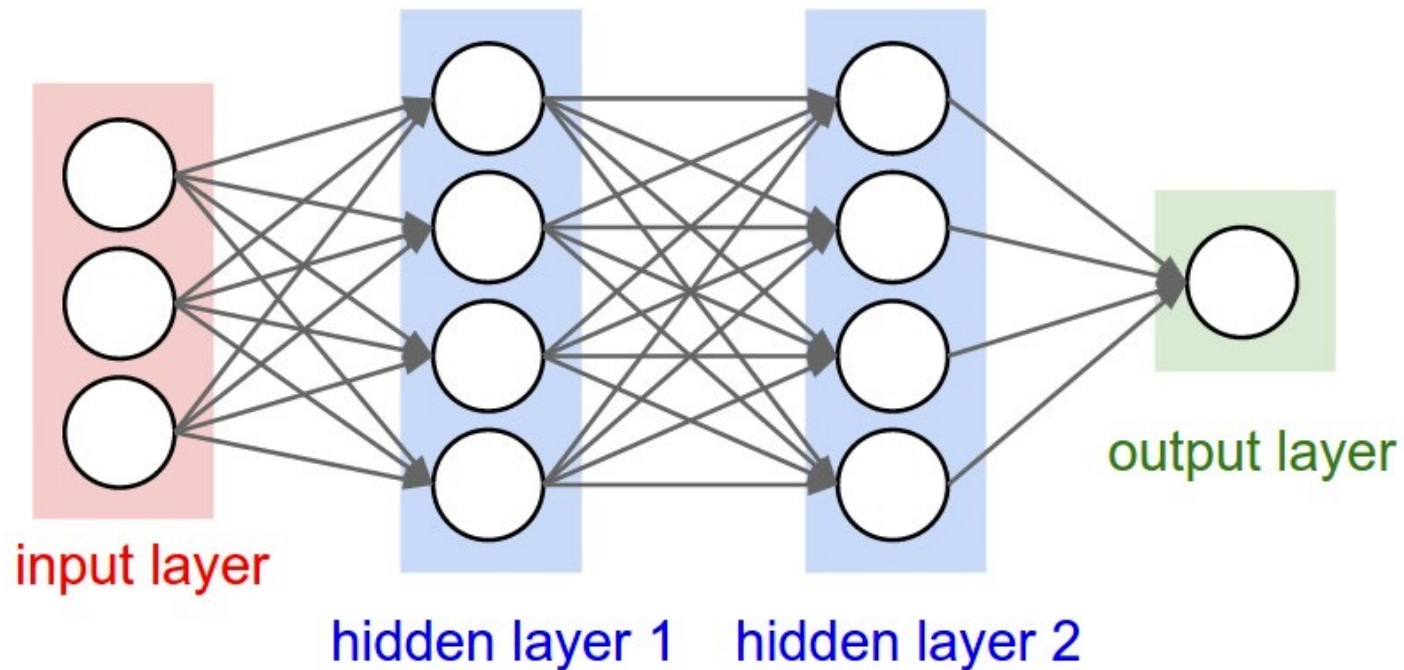


$$f(x) = x^2$$



This simple network can already approximate these functions. Red: resulting network function + other hidden units.

Universal approximation theorem



Simple neural networks can represent a wide range of mathematical functions when given the appropriate parameters. (Cybenko 1989)

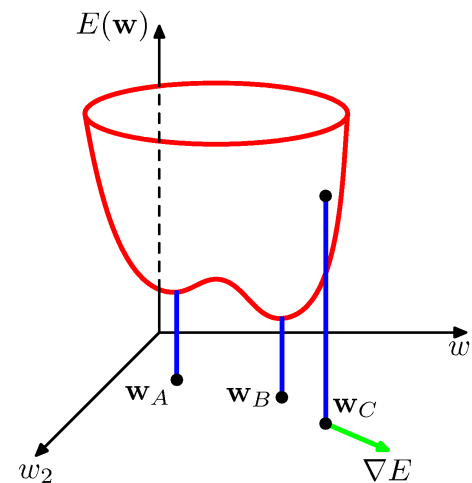
Network training

Given a set of inputs x_i and target vectors t_i we want to minimize the error function:

$$E(w) = \sum_{n=1}^N ||y(x_n, w) - t_n||^2$$

Geometric view of the error function:

$$\frac{\partial E}{\partial a_k} = y_k - t_k$$



Network training

It is not feasible to find an analytical solution for the equation

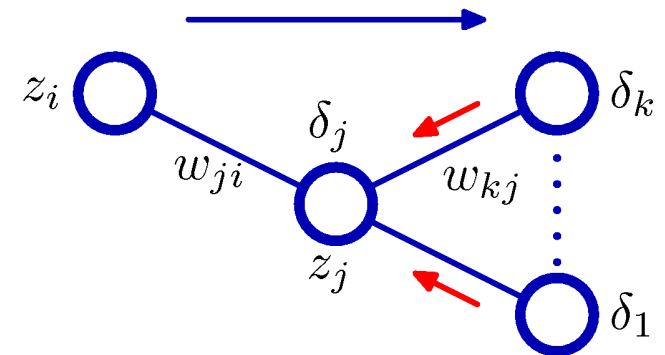
$$\nabla E(w) = 0$$

Instead, we find a solution through iterative steps:

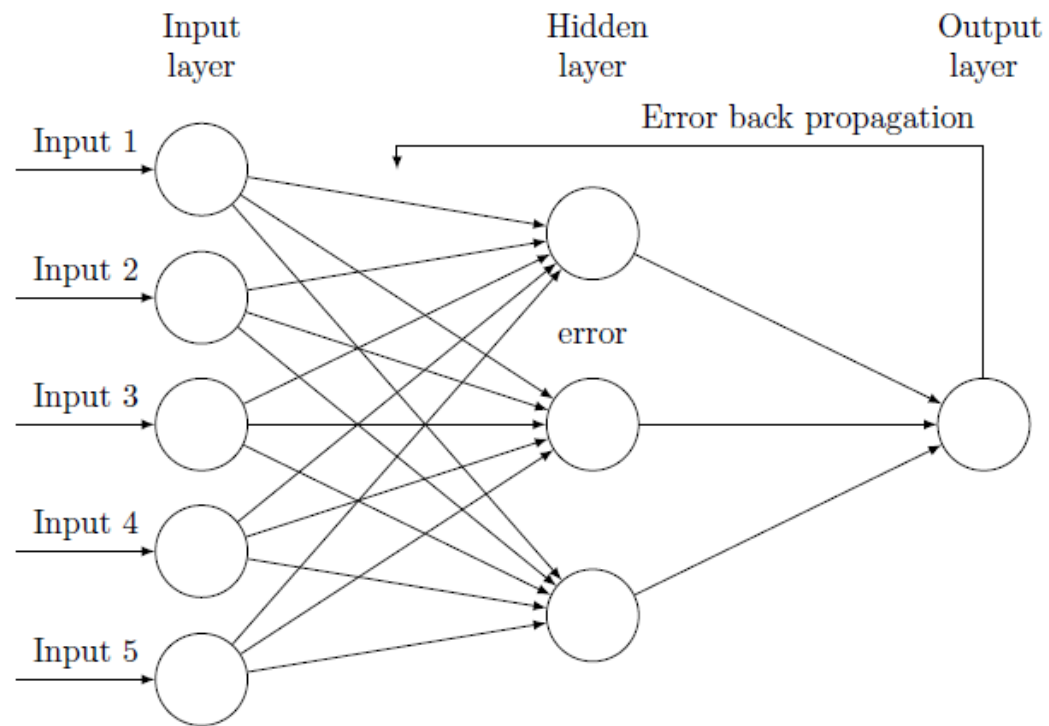
$$w^{(v+1)} = w^{(v)} + \Delta w^{(v)}$$

Error backpropagation principle

- 1) Apply an input vector and forward propagate through the network to find the activation of all the hidden and output units
- 2) Evaluate the $\delta_k = \frac{\partial E_m}{\partial a_j}$ for all of the output units
- 3) Backpropagate the errors to obtain δ_j for each of the hidden units
- 4) Evaluate the derivatives $\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$



Error backpropagation principle



Overfitting

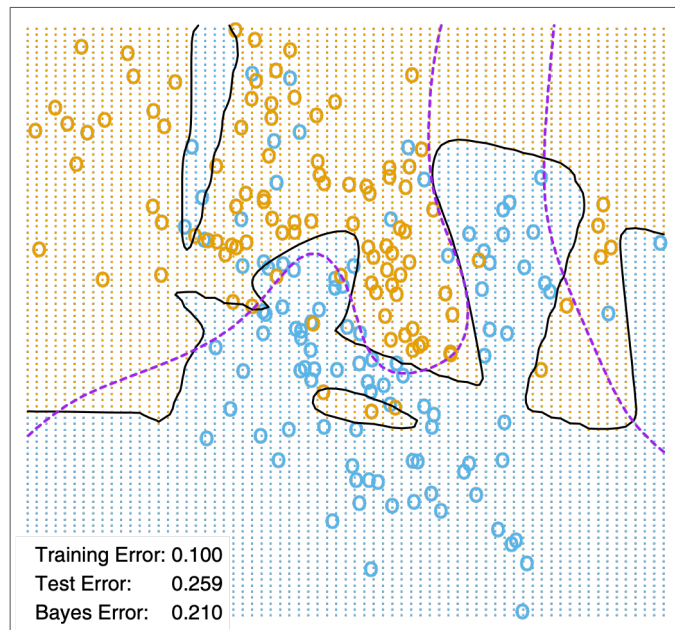
Often neural networks have many parameters and optimising the error function will lead to overfitting.

$$R(\Theta) + \lambda J(\Theta)$$

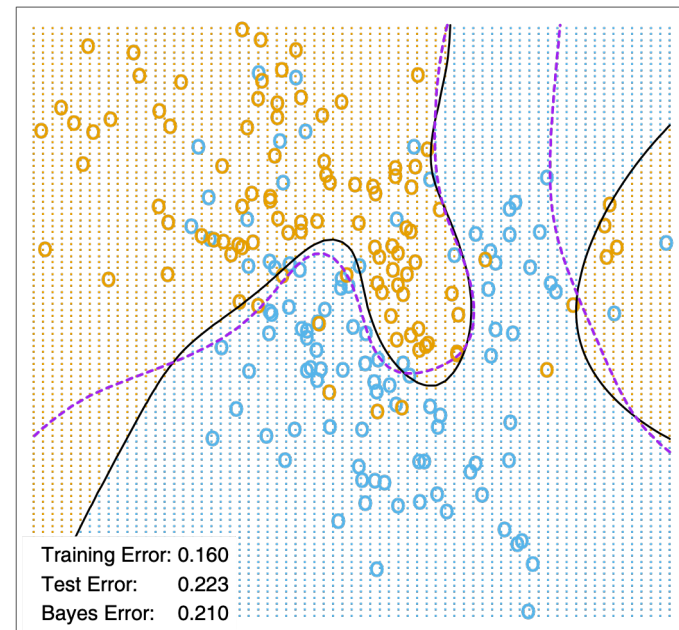
$$J(\Theta) = \sum_{km} \beta_{km}^2 + \sum_{km} \alpha_{km}^2$$

Overfitting

Neural Network - 10 Units, No Weight Decay



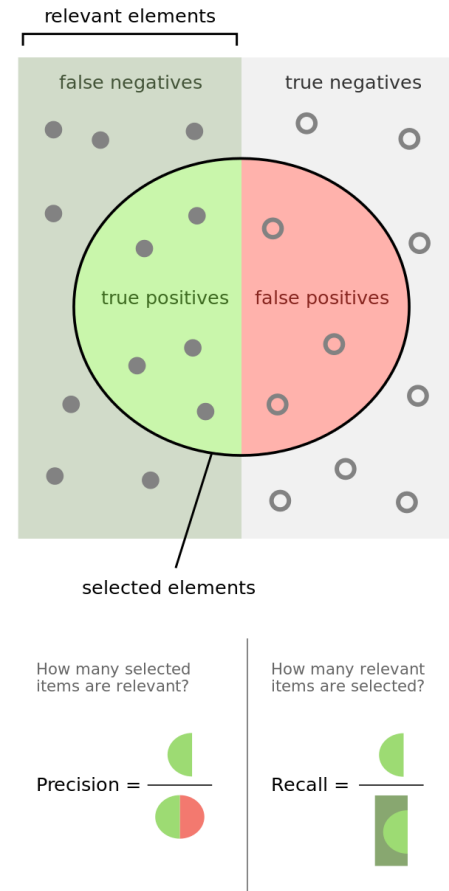
Neural Network - 10 Units, Weight Decay=0.02



Detection: precision vs recall

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$



References

- Christopher Bishop, Pattern Recognition and Machine Learning [[pdf](#)]
- Richard Duda, David G. Stork, Peter E. Hart, Pattern Classification (online pdf copies are available)
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman, The Elements of Statistical Machine Learning (Springer)

Overview of ML books [[web](#)]